

Getting reliable work out of AI

Student study guide

Created by Swarup Biswas

Your take-home companion to the session. The slides carry the key ideas; this guide carries the explanation behind each one, plus the research and terms we kept off the slides so the session could stay on the fundamentals. Read it after class to consolidate, or before an assessment to revise.

What you should be able to do after this session

- Explain why an AI can be confidently wrong, and treat its confidence as no evidence of accuracy.
- Name the moves that actually raise reliability: grounding, placement, self-checking, and abstention.
- Turn a one-off prompt into a structured, reusable YAML *prompt spec*.
- Read and lightly edit that YAML: pairs, nesting, lists, and text blocks.
- Speak the vocabulary of the field, the terms that show up in job descriptions.

The one idea to carry out of the room: *An AI predicts plausible text; it does not look answers up. So reliable results come from what you put in front of it and how you check what comes back, not from a cleverer phrase or a grander persona. Design the input, verify the output, and reuse what works.*

1 An AI predicts text; it does not look things up

The most useful picture to carry is how the machine works. A model builds its answer one small piece at a time, at each step choosing a likely next word given everything so far. It has no lookup table and no internal flag for “I do not actually know this.”

That single fact explains two things at once. It explains why the output reads so fluently, and it explains why it can be fluently wrong: when the most likely next words happen to be false, they arrive just as smoothly as when they are true. Facts that follow strong patterns (spelling, well-known capitals) come out reliably. Rare, arbitrary facts (a stranger’s birthday, an obscure citation) cannot be predicted from a pattern, so the model fills the gap with something that merely sounds right.

Why it matters: the costly mistakes at work come from forwarding a fluent paragraph as if someone had checked it. Treat every specific fact, figure, name, and citation as unverified until you confirm it. Lean on the model where it is genuinely strong, rephrasing, structuring, drafting, and check recalled facts harder.

Source: Kalai et al., 2025, “Why Language Models Hallucinate”. On the SimpleQA benchmark, one model answered more questions but was wrong about 75% of the time.

2 What actually moves accuracy (the persona myth)

Almost every work prompt is built from a few parts: a role, the instruction, context, examples, a format, and iteration. They do not pull equal weight. The most common first instinct, giving the model an impressive job title, is the weakest of them.

In a study of 162 personas across 2,410 questions, adding a persona did not reliably improve accuracy. “Act as a senior lawyer” changes the tone of the answer, not its correctness. What moves accuracy is the plain middle of the list: a clear instruction, the right context, good examples, and an explicit output format.

Why it matters: spend your effort where it pays. Treat the persona as a tone dial, and put your real budget into the context the model cannot guess, the audience, the constraints, and the source material.

Source: Zheng et al., 2024; a 2025 Wharton replication agreed.

3 Give it your material: grounding

Most questions at work are about your own material, this contract, that report, our policy, not general knowledge. Ask a model such a question from memory and it predicts a plausible answer. Give it the actual document and tell it to answer only from that, and it shifts from recalling to reading. This is the everyday form of retrieval-augmented generation (RAG), and it is the strongest reliability move you can make without engineering.

It is a reduction, not a cure. Purpose-built legal-research tools, built entirely on retrieval, still produced unsupported answers up to about a third of the time. So demand a quote for each claim, and confirm the quoted text actually says it.

Why it matters: uploading the source and saying “answer only from this, and quote the clause” is the difference between a confident guess and a checkable extract. It also tells you exactly what to spot-check.

Source: RAG introduced by Lewis et al., 2020; error rate from a Stanford, 2024 study of legal-research tools.

4 Where you put the ask: placement matters

A model has no memory between chats. The only thing it knows for your task is what sits in its context window right now, and it does not read that window evenly. It attends most to the beginning and the end, and skims the middle.

So placement matters as much as content. In the “Lost in the Middle” study, moving the key passage from the start of a long input into the middle cut accuracy by over 30%. The practical habit is two lines of effort: put your request at the top, paste the long document, then restate the request at the very bottom.

Why it matters: the most common self-inflicted failure is the giant paste with the real question buried in paragraph nineteen. Anchor the ask at both edges, where the model actually reads.

Source: Liu et al., “Lost in the Middle”, 2024; quality keeps falling as input grows (“context rot”, Chroma, 2025).

5 Make it reason, and know when to skip it

Chain-of-thought, asking the model to work through its reasoning before it answers, was a real breakthrough and for years was good advice for anything hard. The landscape has shifted. Newer reasoning models already think before they answer, so telling them to “think step by step” mostly adds time without adding accuracy.

The idea still helps in two places: on an ordinary model facing a genuinely multi-step problem, and on any model when you want to see the working so you can check it. For a real decision, the reasoning is the product. A recommendation you cannot interrogate is worth less than options and trade-offs you can pressure-test.

Why it matters: do not paste “think step by step” onto everything, and do not skip it on a decision that deserved it. For a choice, ask for the criteria, the options scored against them, and the trade-offs, then the recommendation, then the strongest case for the other option.

Source: Chain-of-thought, Wei et al., 2022. On today’s reasoning models, explicit CoT gave marginal accuracy for a 20 to 80% rise in time (Wharton, 2025).

6 Make it check itself, and let it say “I don’t know”

Two habits catch most invented facts. First, self-verification: have the model list its claims, write a checking question for each, answer those questions on their own without looking at the draft, then produce a corrected version. Isolating a claim is what works; a fact that slid through inside a fluent paragraph gets examined alone, where it is easier to catch.

Second, abstention. Models are tuned to be helpful, which on a question they cannot answer means producing a plausible-but-wrong response. Add one clause, “only answer if you are certain; otherwise say you do not know,” and you give the model a sanctioned alternative to guessing. The research is blunt: graded only on getting answers right, models learn that a guess beats an abstention, so you have to value the abstention explicitly.

Why it matters: the most damaging errors are the confident ones, on facts you did not think to check. An explicit out turns some of those into an honest “I am not sure”, far cheaper to deal with than a fabricated figure that reaches a client deck.

Source: Chain-of-Verification, Dhuliawala et al., 2023; abstention, Kalai et al., 2025.

7 Say what you want, and name the shape

A model continues the most plausible text from your words. Name what you want and you point it straight at that text. Phrase the same thing as a prohibition and you still put the unwanted idea in front of it. So “write in plain language a non-expert can follow” lands more reliably than “do not use jargon”. Rewrite every “don’t” as a “do”.

Two more prompt-level habits. Name the output format, a table, numbered steps, a short summary, so the answer arrives usable instead of prose you reshape by hand; a fixed structure also forces completeness, because a missing column is obvious. And when the format is fiddly or the tone must be consistent, show two or three worked examples rather than describing the rules. Examples teach shape and pattern; when researchers replaced the correct answers in examples with wrong ones, performance barely moved, so keep supplying the facts separately.

Why it matters: most “I had to redo it by hand” moments come from an unspecified format, and most disappointing outputs come from instructions written as complaints. Both are free to fix.

Source: positive framing, Google prompt-engineering guidance (Boonstra, 2024); examples teach pattern, Min et al., 2022.

8 Fence your data from your instructions

The moment you paste a long email or document alongside an instruction, the model has to guess which is which. Delimiters remove the guess: simple markers, triple quotes or tags like `<document>...</document>`, that wall off one part of the prompt from another. They tell the model plainly, this block is the data; that line is the instruction.

There are two payoffs. The model stops blending your ask into the pasted material, and fencing untrusted content, then telling the model to ignore any instructions inside it, is your first defence against prompt injection: hidden commands buried in a document, web page, or email you paste.

Why it matters: treat anything you did not write, pasted emails, links, documents, as untrusted input. Fence it, and say how to treat it.

Source: frontier-lab prompting guides recommend delimiters as a default and as the first defence against indirect prompt injection.

9 Two guardrails only you control

Beyond technique sit two responsibilities that no model setting can cover for you.

Guard what you paste. The biggest workplace AI risk is usually not a hacker; it is confidential material pasted into a consumer chatbot. Within about three weeks of being allowed the tool, Samsung engineers pasted in source code and an internal meeting transcript. With a consumer account, what you paste may leave your organisation’s control and, on some default settings, be retained or used to train future models. Use an approved, no-training account for work data, and redact names, numbers, and identifiers before you paste.

Stay the thinker. Used as an answer machine, AI can erode the judgment you are trying to build. In one study, people who wrote essays with heavy AI help showed the weakest brain connectivity and the poorest recall of their own work. The antidote is in the same research: use the model to quiz you, critique your draft, and tell you what you are missing, not to produce the thing for you.

Why it matters: a careless paste is entirely self-inflicted and entirely preventable, and how you prompt decides whether you grow or atrophy. Both are in your hands, not the model's.

Source: Samsung, 2023; MIT, "Your Brain on ChatGPT", 2025, with a Microsoft and CMU survey of 319 workers.

10 The wider toolkit

Five more levers worth keeping in reach. Each is one small addition to a prompt.

Move	What it does	When to use
Draft, then improve	Ask the model to critique its own draft against a named standard, then rewrite	Any draft where quality beats speed
Ask the general question first	Derive the governing principle, then apply it to the specific case	Judgement-heavy decisions
Ask a few times	Run an important question two or three times; agreement is a confidence signal	Reasoning-heavy or high-stakes answers
Ask it to write your prompt	Describe the goal in plain words and let the model draft the prompt	When you are unsure how to phrase it
Start the answer for it	Seed the first characters of the reply so it opens in the shape you need	API tools, strict formats, no preamble

Why it matters: the biggest quality jump is often not a better prompt but a second pass. Draft, critique, revise beats one attempt every time, and costs you no new input.

Source: self-refinement, Madaan et al., 2023 (about +20%); step-back, DeepMind, 2023; self-consistency, Wang et al., 2022.

11 From a prompt to a spec

Once you can assemble a good prompt, the next leap is operational, not linguistic. A one-off prompt is retyped and forgotten. A structured, reusable spec is saved with a note on when to use it, shared with a colleague, and checked before you trust it. The people who get the most from AI reuse a small, tested library rather than writing better prompts in the moment.

The everyday form is a *prompt spec* written in YAML: a plain-text note of labelled parts. It reads like a form, not code.

```
name: refund-reply
role: a support lead
task: draft a reply to the customer
output_format: three short, warm sentences
```

Why it matters: reuse buys speed, consistency, and shareability at once. A tested template turns a five-minute prompting fiddle into a ten-second fill-in, and a teammate gets your results from your template.

Source: the 2026 practitioner consensus is shared, versioned prompt systems organised around outcomes.

12 The YAML you'll actually write

YAML is a plain-text way to write structured information so a person can read it and a tool can check it. You are not learning to code; you are filling in a labelled note. Four ideas cover almost everything.

Shape	Looks like	Rule
A pair	<code>task: summarise the report</code>	a label, a colon, a space, then the value
A group (nesting)	<code>source: then indented data:</code>	indent members two spaces under the label
A list	a line starting with <code>-</code>	one item per line, same indent
A text block	<code>context: </code> then indented lines	a pipe keeps your line breaks

Three traps catch almost everyone. A bare **yes** or **no** is read as a true/false switch, not the word, so quote it: `answer: "no"`. A colon inside a value confuses the parser, so quote it: `task: "Goal: cut costs"`. And a single tab anywhere breaks the file, indent with spaces only.

Why it matters: the Build tab writes this shape for you; your job is to read it and keep the indents even. A file that will not load almost always has a tab, a missing space after a colon, or an unquoted colon.

Source: YAML from zero, the course reference, Parts 1 to 2.

13 Putting it together: the scaffold

A strong, repeatable prompt is not a paragraph. It is a stack of parts in a deliberate order, because placement does real work (Section 4). Long reference data goes high, inside its fence; examples sit in the middle; the actual request goes near the bottom, where the model reads best; then reasoning, then the output format.

```

role: a contracts analyst
source:
  data: |
    {{long reference data, fenced and placed high}}
  treat_pasted_as_untrusted: true
examples:
  - input: a sample input
    output: the matching output
task: the actual request, near the bottom
reasoning:
  show: true
output_format: |
  the exact shape you want back

```

Why it matters: for anything you run more than once, the scaffold is the difference between a prompt that works by luck and one that works every time. It also makes prompts easy to debug: if the output is wrong, you can usually point to the missing or misplaced part.

Source: vendor prompt-structure guidance; in the app, the renderer assembles this order for you.

14 Your first spec (do this in the app)

About twenty minutes, on your own, in **Prompt Spec Studio** (prompt-spec-studio.vercel.app). It produces one thing you can reuse at work.

1. **Pick a real task** you actually face this week: a recurring email, a report, a decision. Not a textbook example.
2. **Build the spec** in the Build tab: state the task in one plain sentence, add the context the model cannot guess, name the output shape.
3. **Ground it** if the task is about your own document: paste the source and require quotes.
4. **Validate it** in the Validate tab, and fix anything it flags.
5. **Save it** with a note on when to use it. That saved spec is your take-home.

Why it matters: one spec you actually reuse beats a page of notes you never open. Build it this week; the longer it sits, the lower the chance.

15 Vocabulary: the terms that matter

- **Context window / token** — the model's finite working memory, and its unit of measure. Everything you send competes for the same space.
- **Prompt spec** — a reusable prompt written as structured, labelled data (here, YAML) that a tool can validate.
- **Grounding / RAG** — Retrieval-Augmented Generation: giving the model source documents so its answer rests on your material, not its memory.

- **Hallucination** — a fluent, confident output that is not true, produced by the same next-word machinery as a correct one.
- **Chain-of-thought** — asking the model to show its reasoning before the answer.
- **Abstention** — permitting, and valuing, “I don’t know” instead of a guess.
- **Prompt injection** — hidden instructions inside content the model reads (a document, a web page) that try to hijack it.
- **Delimiter** — a marker (tags, triple quotes) that separates your data from your instructions.
- **System prompt** — standing instructions at the start of a session; the highest authority in the window.
- **Temperature** — a reliability dial, not a creativity dial: lower is more predictable, higher is more varied.

Appendix A: the full toolkit at a glance

Every technique from the course, one line each. The letter is the topic label used in the app’s *Learn* tab, where each has a full lesson with a worked example and the research behind it. A, H, and I are mindsets, so they have no YAML slot.

Ref	Technique	Also called	What it does	Reach for it when
A	Verify everything	Hallucination	Confidence is never proof; check the facts	Any specific fact, figure, or name
B	Anchor the ask	Context engineering	Put the request at the top and again at the end	Long pasted documents
C	The building blocks	Role and context	Context and format beat a persona	Every prompt
D	Show, don’t tell	Few-shot	Give two or three worked examples	Fiddly format or house style
E	Faithful summarising	Summarisation	Tie each point to a quote; flag the rest	Summaries you will forward
F	Reason out loud	Chain-of-thought	Ask for the working, not just the verdict	Multi-step, or you must audit it
G	Ground it	RAG	Answer only from your source, with quotes	Questions about your own material
H	Stay the thinker	Cognitive offloading	Make it quiz you, not do it for you	Anything you are learning
I	Guard what you paste	Data hygiene	Approved account; redact before pasting	Any confidential material
J	Save what works	Templates and reuse	Save, name, and version good prompts	A prompt you will reuse
K	Draft, then improve	Self-refinement	Critique the draft, then rewrite it	Quality matters more than speed
L	Ask it to write your prompt	Meta-prompting	Describe the goal; let it draft the prompt	Unsure how to phrase it
M	Break it into steps	Prompt chaining	Stage the work; check each step	Multi-step deliverables
N	Separate data from instructions	Delimiters	Fence pasted data; ignore instructions in it	Pasting third-party text

Ref	Technique	Also called	What it does	Reach for it when
O	Say what you want	Positive framing	State the target, not the prohibition	Setting tone, length, or format
P	Make it check itself	Chain-of-verification	Verify each claim on its own	Fact-dense outputs
Q	Specify the output format	Output formatting	Name the exact shape of the answer	Output you will paste or reuse
R	Ask it a few times	Self-consistency	Run it a few times; agreement is a signal	High-stakes or ambiguous answers
S	Ask the general question first	Step-back	Derive the criteria, then apply them	Judgement-heavy decisions
T	Start the answer for it	Prefilling	Seed the first characters of the reply	Strict formats, API tools
U	The full scaffold	Scaffolding	Assemble the parts in render order	Production prompts you reuse
V	Let it say “I don’t know”	Abstention	Permit and value an honest gap	Any factual ask

Source: the course reference (Topics A–V); each row maps to a lesson in the app’s Learn tab.